

Jonathan Clem — Video Engineering Profile

Targeting: Software Engineer, Video Algorithm Systems (L4) — Netflix (USA, Remote)

Summary

Thirteen years of hands-on video engineering across the full capture-to-delivery pipeline on mobile and embedded platforms, with production C++ experience in audio/video processing engines and image capture SDKs. Named inventor on a US patent for real-time video production protocols. Shipped custom H.264 encoding paths optimized for latency, bandwidth, and perceptual quality on hardware-constrained devices — work that required deep understanding of NAL unit structure, rate control, and hardware-accelerated codec internals. Built and led the engineering team behind FiLMiC Pro, the most widely used professional video capture app on iOS, as CTO of Cinegenix. Fluent in ffmpeg since the early 2000s; began career as a video editor and on-set DIT before moving into engineering, bringing a practitioner's intuition for compression quality that predates my first line of code.

Patent

US20150350041A1 — Protocols & Mechanisms of Communication Between Live Production Server and Mobile or Remote Clients Inventor: Jonathan Clem, Dean Bisogno | Assignee: Paladin Innovators | Filed: May 30, 2014

This patent covers the WebSocket-based command/response protocol I designed for coordinating real-time video production between a centralized encoding server and thin mobile clients. The system addressed how to expose full production-switcher functionality (camera selection, encode state control, asset routing, hardware control) to mobile devices without requiring workstation-class compute on the client, while keeping all connected clients synchronized through server-pushed state updates.

Low-Latency Video Streaming — Paladin Innovators (2012–2014)

While working at Paladin Innovators, I designed and shipped a system that streamed live

video from iPhones directly into a production video switcher over point-to-point Wi-Fi. The key constraint was latency: this was used for live multi-camera switching, so the video feed from each iPhone had to arrive at the switcher with sub-100ms glass-to-glass latency — at 720p60, over 2012-era Wi-Fi hardware.

Achieving this required:

- **Custom NALU packaging:** Structuring H.264 NAL units to minimize decode-side buffering. Standard container overhead and segmentation strategies added unacceptable latency, so I built a transport layer that delivered slices as close to real-time as the network allowed.
- **Hardware-accelerated encoding tuning:** Working within the constraints of Apple's VideoToolbox (and its predecessors on early A-series silicon) to configure the encoder for lowest-latency output — disabling B-frames, tuning GOP structure, and managing rate control to avoid network-induced stalls.
- **Point-to-point transport:** Bypassing CDN/HLS infrastructure entirely in favor of direct device-to-server delivery over local Wi-Fi, which eliminated the dominant source of latency in conventional streaming architectures.
- **ffmpeg integration:** Used ffmpeg extensively on the server/switcher side for transcoding, muxing, and stream analysis throughout development and production deployment. ffmpeg has been a core tool in my workflow since my pre-engineering days as a video editor and DIT.

This was production-deployed at live events including multi-camera church broadcasts and corporate productions.

Custom H.264 Encoder — FiLMiC Remote / Cinegenix (2015–2019)

As CTO of Cinegenix (FiLMiC Pro), I designed and shipped a streaming video solution for FiLMiC Remote that let a director monitor and control a remote iPhone camera's live feed in real time. This required building a custom H.264 encoding path — not using AVFoundation's standard recording pipeline — specifically optimized for:

- **Lowest achievable latency:** The standard AVCaptureSession → file recording path introduces buffering at multiple stages. I bypassed this by working directly with VideoToolbox compression sessions, producing H.264 NAL units that could be packetized and transmitted frame-by-frame with minimal encode-to-network delay.

- **Product-optimized NALU structure:** Tailored the NAL unit output for the specific decode constraints of the receiving client. This meant controlling SPS/PPS emission frequency, slice sizing, and IDR interval to balance error resilience against bandwidth and latency.
- **Bandwidth-adaptive rate control:** The encoder dynamically adjusted bitrate targets based on network throughput estimation, prioritizing frame delivery continuity over peak quality — the right tradeoff for a live monitoring use case.

This work required reading and reasoning about the H.264/AVC spec at the bitstream level — not just configuring high-level API parameters, but understanding how macroblock-level encoding decisions affected decode latency, error propagation, and perceptual quality under constrained bandwidth.

Camera Frameworks & Video Pipeline Expertise

FiLMiC Pro — CTO, Cinegenix (2014–2019)

Led a complete ground-up rewrite of FiLMiC Pro in Swift 2.0 (shipping with iOS 9), architecting the full AVFoundation capture pipeline: camera device management, manual exposure/focus/white balance control, real-time audio metering, and high-bitrate H.264/HEVC recording with configurable resolution, frame rate, and codec parameters. FiLMiC Pro was the #1 video app on the App Store for multiple years and was used to shoot segments of feature films, documentaries, and broadcast journalism.

Integrated OpenCV for real-time image analysis within the capture pipeline — used for on-device frame inspection and processing tasks that required computer vision capabilities beyond what AVFoundation provided natively.

Built a custom H.264 encoder (separate from the FiLMiC Remote streaming encoder described above) for the core FiLMiC Pro recording path, giving fine-grained control over encoding parameters that AVFoundation's high-level recording APIs did not expose.

Built a proof-of-concept ProRes software encoder for iPhone (achieving ~5-6 fps with ProRes Proxy/LT on contemporary hardware) — demonstrating the gap between software and hardware-accelerated codec performance that later became relevant when Apple shipped hardware ProRes on A15+.

Designed and shipped the C++ audio capture and processing engine used in FiLMiC Pro, handling real-time audio metering, monitoring, and synchronized A/V recording.

CinePro (2012)

Independently built CinePro, ranked #1 in App Store video apps for 2012. Full manual camera control app built on AVFoundation with custom video recording pipeline.

Truepic — Senior iOS Engineer / SDK Tech Lead (2019–2021)

Tech lead on Truepic's image capture, sign, and validate SDK for iOS — a cryptographic content provenance system that verifies photos and videos are unmanipulated from the moment of capture. The SDK was built entirely on underlying C++ libraries that I worked on and maintained daily, with Swift/Objective-C bridging layers for the iOS integration surface. This included the capture pipeline (deep AVCaptureSession integration to embed provenance metadata before any post-processing), cryptographic signing, and server-side validation flows. The pipeline design has direct parallels to content integrity and watermarking problems in streaming video delivery.

DoubleTake

Contributed to FiLMiC's DoubleTake app, which enabled simultaneous multi-camera recording on iPhone — requiring management of multiple concurrent AVCaptureDevice inputs, synchronized frame timing, and independent codec configuration per stream.

Pre-Engineering: Video Editor & DIT

Before transitioning to software engineering, I worked as a video editor and on-set Digital Imaging Technician (DIT) on film and broadcast productions. During this period I became deeply fluent in ffmpeg and Apple Compressor, building custom compression profiles to maximize perceptual quality within extreme bitrate constraints — most notably squeezing a weekly hour-long production into H.264 video conforming to the iPod Video standard format (baseline profile, 320×240/640×480, ~1.5 Mbps). This hands-on compression work in the early 2000s gave me an intuitive understanding of quality-bandwidth tradeoffs years before I started writing encoders.

Why This Background Matters for Netflix Video Algorithm Systems

The Netflix Encoding Technologies team works on improving quality, reducing bandwidth, and scaling video processing. My career has been spent solving these same problems under tighter constraints: mobile hardware, real-time latency requirements, battery budgets, and unpredictable wireless networks.

Codec internals, not just APIs. I've written custom H.264 encoders, configured hardware encode sessions at the NAL unit level, and built compression profiles in ffmpeg — controlling slice structure, reference frame management, and rate control to meet specific latency and quality targets. This is the same domain as Netflix's per-title encoding optimization, applied in a real-time context.

C++ in production. Built the C++ audio capture/processing engine for FiLMiC Pro and served as tech lead on Truepic's C++-based image capture and cryptographic signing SDK. This is hands-on, daily C++ work in performance-sensitive media pipelines — not wrapper code.

Computer vision and image processing. Integrated OpenCV into FiLMiC Pro's real-time capture pipeline for on-device frame analysis. This overlaps directly with the image and signal processing background the role calls for.

ffmpeg fluency. Twenty years of ffmpeg usage spanning professional video editing, custom encoding profiles, transcoding pipelines, and stream analysis. Comfortable reading ffmpeg source and reasoning about codec behavior at the filter/demuxer level.

Quality under constraint. Shipping broadcast-quality video from an iPhone over Wi-Fi to a production switcher at sub-100ms latency — in 2012 — required making every bit count. That intuition for quality-bandwidth tradeoffs is directly applicable to the work Netflix describes: making streams smoother, crisper, and more efficient.

Full-pipeline thinking. From sensor readout through ISP processing, color space management, codec configuration, transport, and decode — I've built or modified every stage of the video pipeline. This end-to-end understanding is valuable when optimizing any single stage, because encoding decisions don't exist in isolation.

Shipping products, not just research. Every system described above was deployed to real users in production. FiLMiC Pro served hundreds of thousands of professional and prosumer filmmakers. The Paladin system was used at live events. I know what it takes to move from prototype to product — the Netflix job description emphasizes taking innovations "from paper to product."

Contact

Jonathan Clem [johnnyclem](#) on GitHub | Buda, TX